



OpenAI

ANTHROPIC

AI Best Practices

A Unified Guide



Principles



Prompting
Best Practices



Model
"Thinking"



Tools and
Agents



Document
Processing



Safety and
Guardrails



Testing and
Evaluation



Advanced
Coding Practices



Common Failure
Modes and
Anti-Patterns



Trade-offs



Production
Readiness



Continuous
Improvement



Constitutional AI

Harmlessness	✓
Honesty	✓
Helpfulness	✓

System

User

Assistant

Context



- Based on OpenAI's integration with Anthropic's official documentation

This document consolidates and systematizes all the core official articles of the two companies on "How to Use AI Well". The structure is organized according to **the underlying principles** rather than by the company, with the aim of finding out the common essence of the two companies, the trade-offs in different aspects, and the specific practices that can be directly applied in different scenarios.

0. Sources

OpenAI

- «A Practical Guide to Building Agents»
- «Prompt engineering»
- «GPT-5 prompting guide»
- «GPT-5.1 prompting guide»
- «Prompt guidance»
- «Reasoning best practices»
- «Prompt engineering best practices for ChatGPT»
- OpenAI Academy prompts engineering resources

Anthropic

- Prompting best practices (Claude API documentation, active version core reference)
- «Prompt engineering overview»
- «Building effective agents» (Anthropic Blog)
- «Building agents with the Claude Agent SDK»
- «Claude Code best practices»
- «Writing effective tools for agents»
- «Effective harnesses for long-running agents»
- «Equipping agents for the real world with Agent Skills»
- «Define success criteria and build evaluations» (Strengthen guardrails)

1. Underlying principles: why prompt to write like this

What the two companies repeatedly emphasize in all their documents are three facts that cannot be bypassed.

1.1 The model cannot read minds. OpenAI's "clear instructions" and Anthropic's "clear and direct" say the same thing: the model doesn't know the implicit assumptions in your head. Anthropic gives a criterion - show your prompt to a colleague who doesn't know the context of the task, and if he gets confused, so will the model.

1.2 Think of the model as a "new colleague who is smart but lacks organizational background". This is an analogy used by two companies. No matter how smart your new colleagues are, they don't know your naming conventions, business processes, customer preferences. So **information density is more important than passwords**: instead of repeatedly emphasizing "important! It is better to write down the facts and constraints it needs to know directly.

1.3 Prompt engineering is iterative, not one-time. Three prerequisites must be completed before the "reminder":

1. Clear success criteria (what is success?))
2. Empirical test methods (how to measure?))
3. A first draft tip (first to optimize)

Without evaluation, all "tuning" is based on feeling. Anthropic puts this at the beginning of all prompt engineering documentation, and OpenAI makes it a separate chapter of the six strategies for "systematic testing changes."

2. Generic prompting techniques (model-agnostic)

This chapter is the core technology that applies to both GPT and Claude, whether it is an API or a chat interface.

2.1 Clear and direct instructions

Say **"what to do" instead of "what not to do"**. Anthropic makes this clear; OpenAI gives the same advice in format control.

- Weak: Don't answer with markdown.
- Better: Answer with fluent prose paragraphs.

Specify the specific attributes of the output: length, format, structure, style, tone, target audience. The counterexample given by OpenAI is very intuitive - "writing an article about marketing" is far inferior to "writing a 500-word article for small and medium-sized business owners and focusing on content marketing".

Sequential steps are numbered or symbolized columns. When order or completeness is important, break down the instructions into innumerable steps.

2.2 Provide context and motivation

Don't just give instructions, give "why". Original text from Anthropic: Models are smart enough to generalize from explanations.

- Weak: Always answer user questions before asking clarifying questions.
- Better: Answer the user's question first and then ask clarifying questions. Our users tend to churn while waiting, and answering first reduces bounces.

Writing the environment in which the task occurs (who the user is, who to show it, and what purpose) can almost always make the output more accurate.

2.3 Examples (Few-Shot / Multishot)

The two companies agree that the example is to change the leverage that outputs the most stability. Anthropic **suggests 3–5 examples** and emphasizes three points:

- **Relevance:** Examples should be close to real-world use cases
- **Diversity:** Covers boundary situations and the differences are large enough so that the model does not catch irrelevant commonalities
- **Structured:** Separate with `<example>` tags (multiple with `<examples>`) and instructions

OpenAI recommends **few-shot over zero-shot** in the Help Center, and shows how to feed samples to the model using `<product_review>` and `<assistant_response>` pairing.

When not to use examples: When using an inference model (OpenAI's o-series), trying zero-shot first, it is often sufficient; If you add examples, make sure that the examples and instructions are highly consistent, otherwise it will have the opposite effect.

2.4 Structured Markup: XML Tags + Markdown

This is a highly convergent approach between the two companies.

Anthropic uses XML tags as the number one recommendation technique due to the heavy use of XML in Claude's training process. Common patterns:

```
<instructions>...</instructions>
<context>...</context>
<input>...</input>
<examples>
  <example>...</example>
</examples>
```

OpenAI clearly recommends combined usage after GPT-5: Markdown titles divide sections, and XML tags define referable content fragments. A typical developer message is organized in this order:

- **Identity:** The assistant's role, style, and goals
- **Instructions:** What to do and what not to do
- **Examples:** Input and desired output
- **Context:** Private data, relevant information (usually placed last)

Why XML is better than plain text: When prompts are a mix of instructions, examples, user input, and reference material, the model tends to mix them together. Labels are signals to the model "what type of content is this".

Important details (Anthropic): Label naming should be consistent; Nesting is used when there is a natural hierarchy nested (`<documents>` is used to put `<document index="1">`).

2.5 Character Setting

Setting roles in system/developer messages is useful even with just one sentence.

```
You are a helpful coding assistant specializing in Python.
```

An extra reminder from Anthropic: **Don't over-restrain characters** – too narrow role definitions can make the model rigid in edge cases.

2.6 Special Handling for Long Contexts (20k+ tokens)

When you have to bring in a lot of reference material (contracts, codebases, research documents), the location and structure have a huge impact on the effect.

Anthropic's three specific practices:

1. **Long documents are placed at the front**, and queries/instructions/examples are placed at the back. In testing, this adjustment improved response quality by up to 30%.
2. **Multi-document wrapped in XML:**

```
<document index="1">
  <source>q3_earnings.pdf</source>
  <document_content>...</document_content>
</document>
```

3. **Have the model quote the relevant paragraph before** answering. This helps the model focus on really relevant information from the noise.

OpenAI's response is to put content that will be used repeatedly at the top of the prompt and the JSON body of the API request at the top to maximize the cost and latency savings brought by **prompt caching**.

2.7 Controlling the Output Format

Four ways, from coarse to fine:

1. **Clearly tell the model what it wants:** "Use smooth paragraphs" instead of "don't use lists"
2. ****Use XML**:** `<smoothly_flowng_prose_paragraphs>`
3. **Make the prompt style match the output style:** If you use less markdown in your prompt, there will be less markdown in the output
4. **Detailed specification:** Long instructions explain when and in what format (see Anthropic's `<avoid_excessive_markdown_and_bullet_points>` template)

OpenAI's default behavior in the API: The GPT-5 series **does not output markdown by default** in the API (preserving compatibility). To get it to output markdown, you need to add `Formatting re-enabled` or explicit instructions to the first line of the developer message.

2.8 Make the order priority clear

OpenAI Chain (chain of command) :

- `system` / `developer` message: rules and business logic set by the application - like function definitions
- `user` message: Input from the end user - like a function parameter
- `Assistant` message: Content generated by the model

Higher priority instructions override lower priority - this is why putting core rules into developer/system messages resists user prompt injection.

3. Let the model "think": inference control

This is the core differentiation between the two recent generations of models, and it is also where users are most likely to make mistakes.

3.1 Inference Models vs. Non-Inference (GPT) Models

OpenAI gives a very clear dichotomy:

Dimensions	GPT model	Inference Model (O-Series / GPT-5 reasoning)
Analogy	Colleagues with less qualifications	Senior colleagues
Fit	Clearly defined tasks that need to be performed	Complex, judgment-oriented, ambiguous questions
Prompt style	Precise, detailed, and prescribed steps	High-level goals, let it figure it out
Speed/cost	Fast and cheap	Slow and expensive
When to choose	Speed + cost is the most important	Accuracy + reliability is paramount

The most common anti-pattern: Using an inference model method to prompt the GPT model (too general), or vice versa (step-by-step interferes with reasoning).

3.2 Inference depth parameters

Both have knobs for "reasoning strength":

- **OpenAI:** `reasoning_effort` — `minimal` / `low` / `medium` / `high`
- **Anthropic:** `effort` — `low` / `medium` / `high` / `xhigh` / `max`

The two common recommendations:

- Execution-based workflows (field extraction, classification, light transformation) → Start with low effort
- Research-based workflows (long-context synthesis, multi-document review, strategy writing) → medium or high
- Programming with Agent tasks → high or high
- When you see "overthinking" (even simple tasks grinding) → turn it down
- When you see "not enough thinking" (answering complex questions too shallowly) → turn it up, **don't rely on prompts to get around**

Anthropic's clear warning: Claude Opus 4.7 will be executed strictly literally at low intensity and will not "empathetically" generalize instructions. Specify the scope when it needs to be broad ("This format applies to every section, not just the first section").

3.3 Think about the guidance of triggers

If the model thinks too much (which is often caused by large system prompts), you can join the guidance:

Thinking adds latency and should only be used when it will meaningfully improve answer quality – typically for problems that require multi-step reasoning. When in doubt, respond directly.

Conversely, if you want to force thinking:

This task involves multi-step reasoning. Think carefully through the problem before responding.

3.4 Key Prompting Principles for Inference Models (OpenAI)

- **Don't write CoT prompts:** "Let's think step by step" is useless or even harmful to the reasoning model – it is already thinking internally
- **Keep it short and to the point:** high-level goal > detailed steps
- **Zero samples first,** and then add fewer samples if necessary
- **Clear constraints:** "Budget under \$500"
- **Define success criteria:** let it know what is done

4. Tool usage with Agent Build

This chapter is the part of documentation growth in the two years from 2024 to 2026, as agents are the scenarios where the value of the model is truly realized.

4.1 When to build an agent

OpenAI gives three types of high-value scenarios in "A Practical Guide to Building Agents":

1. **Complex decisions:** Requires nuanced judgments, exception handling, and context-sensitive decisions (e.g., refund approval in customer service)
2. **Difficult rules to maintain:** Rulesets become complex and expensive to update (e.g., vendor security reviews)
3. **Large amounts of unstructured data:** natural language understanding, document information extraction, conversational interaction (e.g., insurance claims processing)

The other way around: if a deterministic solution is sufficient, don't use the agent. Agents trade latency and cost for the ability to handle ambiguity.

4.2 The Essential Difference Between Anthropic: Workflow vs. Agent

- **Workflow:** LLMs and tools are orchestrated through **predefined code paths**
- **Agent:** LLMs **dynamically determine** their own processes and tool usage

Anthropic repeatedly emphasizes (this is the core disagreement between them and many framework vendors): **the most successful implementations are simple composable patterns, not complex frameworks**. If an enhanced LLM (with retrieval, tools, memory) can do it, don't use the Agent.

4.3 Five Core Patterns (Anthropic)

1. **Prompt Chaining:** A series of fixed steps, each step output is the next input. Suitable for: translation, generation, review, rewriting, standardization process.
2. **Routing:** Distribute different types of inputs to different specialized prompts/models. Suitable for: customer service classification, simple queries to small models, complex queries to large models.
3. **Parallelization:** Two variants –
 - **Sectioning:** Cutting tasks into independent subtasks and running them in parallel
 - **Voting:** Run the same task multiple times, and merge the results to increase confidence
4. **Orchestrator-Workers:** An LLM dynamically disassembles a task and assigns subtasks to the "worker" LLM. Suitable for tasks whose complexity cannot be predetermined, such as code modifications across multiple files.
5. **Evaluator-Optimizer:** One LLM generates, another evaluates and provides feedback, looping improvements. Suitable for tasks with clear evaluation criteria and valuable iterative improvements (e.g., translation polishing, literary search).

4.4 Single Agent vs. Multiple Agents (OpenAI)

OpenAI's strong recommendation: **first achieve the ultimate in a single agent, and then consider dismantling**. Multi-agent is intuitively more "modular" but comes with coordination and evaluation overhead.

When to dismantle:

- **Complex logic:** When there are a large number of if-then-else branches in the prompt, the template is unsustainable
- **Tool overload:** When the number of tools exceeds 10-15, or when the similarity/overlap between tools causes the model to be wrong

Two multi-agent modes:

- **Manager mode:** A central Manager Agent coordinates multiple specialized agents through tool calls. It is suitable for scenarios where an agent is needed to coordinate the user experience.
- **Decentralized mode:** Agents handoff each other. It is suitable for scenarios where triage and specialized agents are completely taken over.

4.5 Best Practices for Tool Design (Anthropic Engineering Blog)

The quality of the tool determines the ceiling of the agent.

Principle of tool description: Treat tool description as "onboarding instructions for new employees". Explicitly write the context you implicitly bring into (special query formats, term definitions, relationships between resources).

Name it without ambiguity: use `user_id` instead of `user`; Use `customer_email` instead of `email`.

Inputs and outputs should be strictly typed: use a strict data model instead of free text.

Limit tool response size: Claude Code limits tool responses to 25,000 tokens by default. Implement pagination, range selection, filtering, truncation.

Error responses should also be prompted: error messages should clearly state "what should be changed and how" instead of stack tracing.

Anthropic's meta-approach: Optimize the tool with Claude itself. Build an evaluation set and ask Claude to rewrite the

tool description and compare the results—their practice shows that this cycle can lead to dramatic improvements.

4.6 Parallel tool calls

The latest generation model excels in parallel tool calls. To stimulate this behavior:

If you intend to call multiple tools and there are no dependencies between the tool calls, make all of the independent tool calls in parallel. Prioritize calling tools simultaneously whenever the actions can be done in parallel rather than sequentially. However, if some tool calls depend on previous calls to inform dependent values like the parameters, do NOT call these tools in parallel and instead call them sequentially.

4.7 Controlling the "Positivity" of Agents (GPT-5 / Claude 4.6+)

A recurring problem with the new generation model is **over-aggressiveness** - too much exploration, unnecessary scaffolding, too much extra work.

Reduce Motivation (OpenAI's `<context_gathering>` Template):

Goal: Get enough context fast. Parallelize discovery and stop as soon as you can act.

Method:

- Start broad, then fan out to focused subqueries.
- Avoid over searching for context.

Early stop criteria:

- You can name exact content to change.
- Top hits converge (~70%) on one area/path.

Loop:

- Batch search → minimal plan → complete task.
- Search again only if validation fails.

Boost Motivation (OpenAI's `<persistence>` Template):

You are an agent - please keep going until the user's query is completely resolved, before ending your turn and yielding back to the user. Only terminate your turn when you are sure that the problem is solved. Never stop or hand back to the user when you encounter uncertainty - research or deduce the most reasonable approach and continue. Do not ask the human to confirm or clarify assumptions.

Anthropic corresponds to the "Overeagerness" and "Avoid focusing on passing tests" paragraphs. Specifically: Clearly state "don't add extra features unasked, don't refactor code that isn't required, don't add defense code for things that don't happen".

4.8 Tool Preambles for Tool Calls

Having an agent briefly explain what it does and why before calling the tool can greatly improve the user experience for long tasks.

`<tool_preambles>`

- Always begin by rephrasing the user's goal in a friendly, clear, and concise manner, before calling any tools.
 - Then, immediately outline a structured plan detailing each logical step.
 - As you execute, narrate each step succinctly and sequentially.
 - Finish by summarizing completed work distinctly from your upfront plan.
- `</tool_preambles>`

5. Context Engineering

This is the core concept that Anthropic is proposing in the second half of 2025: **more context is not more; The "signal-to-noise ratio" of the context determines the output.**

5.1 Context Rot

When the context window is stacked, the model attention is diluted. The two practices are:

- **OpenAI:** `compaction` API, `prompt caching`, Clear conversation state management
- **Anthropic:** Context awareness (the model knows how many tokens are left), adaptive thinking, memory tools

5.2 Multi-Context Window Workflow (Anthropic)

When tasks span a single context window (such as refactoring a large codebase), **it's critical to get the model back from a clean window.**

Five specific practices:

1. **The first window uses different prompts:** use the first context window to "initialize the agent", set the framework (write tests, create startup scripts), and then iterate on this basis.
2. **Have the model write tests in a structured format:** Let Claude create tests and save them as `tests.json` before starting. Clearly state that "tests cannot be deleted or modified, as this may result in missing functionality or bugs".
3. **Set up quality of life scripts:** Let the model create startup scripts like `init.sh` to gracefully start services, run tests, and run linters.
4. **Prioritize new windows over compression:** Claude 4.5+ is very good at restoring state from the local file system. Give it clear recovery instructions:

```
- Call pwd; you can only read and write files in this directory.
- Review progress.txt, tests.json, and the git logs.
- Manually run through a fundamental integration test before
  implementing new features.
```

5. **Provide validation tools:** As autonomous task time grows, the model needs to validate itself without human feedback (Playwright, computer use, etc.).

5.3 Best Practices for State Management

- **Structured data in structured format:** JSON for test results and task status
- **Progress notes in free text:** progressive recording
- **Track state with git:** git log is a natural state machine
- **Emphasize incremental progress:** Explicitly let the model take a small step at a time, commit, and then continue

5.4 Skills: Progressive Disclosure (Anthropic Open Source Standard)

Skill is an organized folder (containing SKILL.md plus optional scripts, references) that can be discovered and loaded dynamically by the Agent. At the heart of the design is **progressive disclosure**:

- Only the name and description of each skill are put into the system prompt at startup
- The model decides when it needs to load the full content
- This means that the content within the Skill can be infinitely large, as it does not enter the underlying context

When to build a skill:

1. **General Enhancement:** What models are not good at out of the box (generate PDF, Excel, PowerPoint)

2. **Organizational/industry/individual workflows:** Your brand guidelines, customer-specific compliance requirements

5.5 Key components of Agent Harness (Anthropic)

Building an "environment" for a long-distance runner is more important than tuning tips:

- **Initialize the agent:** Establish the environment, write requirements documents, and set up tests during the first runtime
- **Progress files** (like `claude-progress.txt`) + git history: Allow new sessions to quickly understand task status
- **Clear start/end signals:** The model knows what to do and what to finish a session for

6. Guardrails

6.1 Multi-layered defense (OpenAI)

OpenAI emphasizes that "a single guardrail is not enough" – building a multi-layered portfolio:

Guardrail type	Function
Relevance classifier	Block off-topic input ("How tall is the Empire State Building" is blocked by the telecom customer service agent)
Safety classifier	Detect jailbreak / hint injection
PII filter	Prevent accidental disclosure of personal information
Moderation	Hate speech, harassment, and violent content
Tool safeguards	Assess risk for each tool (read vs write, reversible vs irreversible, financial impact)
Rules-based protections	Blacklists, length limits, regulars
Output validation	Brand consistency, content compliance

Optimization Strategy:

1. Focus on data privacy and content security first
2. New guardrails are added based on real failure cases
3. Continuously tune between security and user experience

6.2 Balancing Autonomy and Safety (Anthropic)

Claude Opus 4.6 can take actions that are difficult to undo when unguided. Specific method:

Consider the reversibility and potential impact of your actions.
Encouraged to take local, reversible actions like editing files or running tests, but for actions that are hard to reverse, affect shared systems, or could be destructive, ask the user before proceeding.

Examples of actions that warrant confirmation:

- Destructive operations: deleting files, dropping tables, `rm -rf`
- Hard to reverse: `git push --force`, `git reset --hard`
- Operations visible to others: pushing code, commenting on PRs, sending messages

6.3 Human in the Loop

Two types of triggers (OpenAI):

1. **Failure threshold exceeded:** Multiple consecutive failures to understand user intent → escalate to human
2. **High-risk actions:** sensitive, irreversible, high-stakes (cancellations, large refunds, payments) → forced labor before agent reliability is established

6.4 Prompt Injection Defense

- Treat external content (documents, web pages, user-uploaded files) as **data rather than instructions**
 - Clearly label "This is the reference material" vs. "This is the instruction" with XML tags
 - System-level guardrails detect obvious injection patterns
-

7. Testing and evaluation

This is the section that is put at the forefront by both companies but most often skipped by developers.

7.1 Define the success criteria first

Before writing a prompt, answer:

- What does the "right" look like for this task?
- How to measure? (Automated Metrics / LLM-as-judge / Evaluation)
- How much does it cost to fail?

Anthropic asks for three things in the first section of the prompt engineering overview: **clear success criteria, empirical testing methods, and first draft prompts.**

7.2 Self-evaluation allows the model to self-check (common to both companies)

Have the model check against the criteria before giving a final answer:

```
Before you finish, verify your answer against [test criteria].
```

OpenAI's "self-reflection" template (for zero-to-one app generation):

```
<self_reflection>
- First, think of a rubric until you are confident.
- Create a rubric with 5-7 categories. This is for your purposes only.
- Use the rubric to internally iterate. If your response is not hitting
  the top marks across all categories, you need to start again.
</self_reflection>
```

7.3 LLM as Judge

OpenAI specifically mentions in Reasoning best practices that using inference models to evaluate the output of other models is a high-value use case. For example, when asked o1 to evaluate the quality of GPT-4o's summary, the F1 score improved from 0.12 to 0.74.

7.4 Systemic changes rather than "feels"

After modifying the prompt, you must retest **the full evaluation set**, not just one or two examples. Cause:

- Fixing case A often breaks case B
 - Individual "feeling better" may not be statistically significant
 - The non-deterministic nature of the model makes a single result unreliable
-

8. Special practice in programming scenarios

Code is the most important agent use case today, and both have a lot of targeted documentation.

8.1 Persistent configuration: CLAUDE.md / AGENTS.md

The two invariably recommend **project-level long-term memory**:

- Anthropic: `CLAUDE.md` (project root and subdirectory hierarchical)
- OpenAI Codex: `AGENTS.md`

What is written in it:

- Project conventions (naming, directory structure, style)
- Must-run commands (start, test, linter)
- Key "traps" and anti-patterns
- Team consensus and decision records

8.2 Core prompt templates for programming tasks

Anthropic recommends setting up the Programming Agent:

```
<investigate_before_answering>
Never speculate about code you have not opened. If the user references
a specific file, you MUST read the file before answering. Make sure to
investigate and read relevant files BEFORE answering questions.
</investigate_before_answering>
```

OpenAI (Terminal-Bench prompt) :

```
<exploration>
Before coding, always:
- Decompose the request into explicit requirements, unclear areas,
  and hidden assumptions.
- Map the scope: identify codebase regions, files, functions, libraries.
- Check dependencies: frameworks, APIs, config files, data formats.
- Resolve ambiguity proactively.
- Define the output contract: exact deliverables.
- Formulate an execution plan in your own words.
</exploration>

<verification>
Routinely verify your code works as you progress, especially deliverables.
Don't hand back to the user until you are sure the problem is solved.
</verification>
```

8.3 Avoid "hacking to pass the test"

Anthropic clearly tips:

Please write a high-quality, general-purpose solution using the standard tools available. Do not create helper scripts or workarounds. Implement a solution that works correctly for all valid inputs, not just the test cases. Do not hard-code values or create solutions that only work for specific test inputs.

If the task is unreasonable or if any tests are incorrect, please inform me rather than working around them.

8.4 Avoid over-engineering

One of the most common anti-patterns. Anthropic's clear tips:

Avoid over-engineering. Only make changes that are directly requested or clearly necessary.

- Scope: Don't add features, refactor, or make "improvements" beyond what was asked.
- Documentation: Don't add docstrings or comments to code you didn't change.
- Defensive coding: Don't add error handling for scenarios that can't happen. Trust internal code guarantees. Only validate at system boundaries.
- Abstractions: Don't create helpers for one-time operations. Don't design for hypothetical future requirements.

8.5 Front-end design avoids "AI Slop" aesthetics

Both companies noticed in the second half of 2025 that the models would converge to a "universal AI flavor" when designing on the front end - purple gradients, Inter fonts, cookie-cutter card layouts.

Coping:

- Describe specific visual directions (not just "modern" or "clean"), specific to fonts, color swatches, margins, shadows
- Or let the model suggest 4 different directions for users to choose from before hands-on
- Anthropic has made this bootstrapping a frontend-design skill

8.6 Multi-Window Programming Workflow

See Section 5.2. The key is to allow one session to gracefully hand over work to the next, rather than trying to "get it done with a hammer".

9. Common pitfalls and anti-patterns

Summarize the "don't do this" content in the two documents.

9.1 Contradictory or vague instructions

The new generation model (GPT-5 / Claude 4.6+) **will spend inference tokens to try to reconcile contradictions** instead of picking one at random. OpenAI showed an example of a significant performance improvement after three hidden contradictions in customer service prompts were resolved in the GPT-5 prompting guide.

Practical checks: Regularly give the core prompt library to GPT-5 / Claude to review inconsistencies yourself.

9.2 Over-prompting

The "must! "Very important!! ", "When in doubt, use tool X" - these often lead to **over-triggering** on new models. Anthropic makes it clear: change the strong tone to the normal tone. When working with OpenAI to tune GPT-5, the Cursor team also found that removing emphasis such as "BE THOROUGH" and "FULL PICTURE" allows the model to make smarter decisions about when to explore and when to answer directly.

9.3 Use GPT prompt styles on inference models

Detailed step-by-step instructions are a burden on the inference model. **In turn**, giving GPT models only high-level targets will make it perform crotch-pulling.

9.4 Let the model guess the code

Especially when the user mentions a specific file name, let the model read it instead of guess it – otherwise it creates plausible hallucinations.

9.5 Creating too many temporary files

Claude 4.5+ creates temporary scripts as "scratch paper" as it is programmed. If you don't want this:

If you create any temporary new files, scripts, or helper files for iteration, clean up these files by removing them at the end of the task.

9.6 Finish hastily when the context is exhausted

Let the model know that there is a compression/memory mechanism and don't end it early due to token anxiety:

Your context window will be automatically compacted as it approaches its limit, allowing you to continue working indefinitely from where you left off. Therefore, do not stop tasks early due to token budget concerns. As you approach your token budget limit, save your current progress and state to memory before the context window refreshes.

9.7 Stuff all requirements into the model at once

In multi-step tasks, **it is better to make it clear in advance** than to "make up while walking". The Claude 4.7 documentation clearly states that vague or under-specification requirements are given gradually, which will **reduce** performance and token efficiency.

10. Divergence between the two companies

While 90% of best practices converge, there are a few notable differences.

10.1 Prompt Styles for Inference Models

- **OpenAI:** For the O-series inference model, clearly say "don't use CoT prompts" and "keep it short"
- **Anthropic:** Still encourages models to think, using `<thinking>` labels for structured reasoning; But it also states that "general instructions are better than prescriptive steps"

Essentially in the same direction – give the goal and let the model think for itself – but with a different style of presentation.

10.2 Verbosity 控制

- **OpenAI:** GPT-5 introduces independent `verbosity` API parameters (differentiated from `reasoning_effort`)
- **Anthropic:** Control by prompt command; Claude Opus 4.7 **automatically calibrates the length based on task complexity** by default

10.3 Prefilled Responses

- **OpenAI:** Not directly emphasized
- **Anthropic:** As of Claude 4.6, support for prefilling the last assistant message has **been removed** – this once common technique is now deprecated and moved to a normal imperative prompt

10.4 Markdown Default Behavior

- **OpenAI:** The GPT-5 API **does not output markdown** (compatibility) by default, and it is explicitly enabled when needed

- **Anthropic:** Claude uses markdown by default, but there are very specific guidelines (fluent prose vs. list selection)

10.5 Framework orientation

- **OpenAI:** Provides a more complete productization stack such as Agents SDK, ChatKit, and Agent Builder
- **Anthropic:** Prefer "simple composable patterns" with reservations about complex frameworks ("Complex frameworks are not used for successful implementation")

11. Quick check of core prompt templates

Below are industrial-grade tips that can be applied directly (from two official documentation).

11.1 General Clear Instruction Template

```
# Identity
You are [role], specialized in [field]. Your goal is to [target].

# Instructions
- [Specific Behavior 1, affirmative statement instead of negation]
- [Specific Behavior 2]
- [Boundary Conditions and Exception Handling]

# Examples
<example>
  <input>...</input>
  <output>...</output>
</example>

# Context
[Proprietary data or reference materials]
```

11.2 Long Document RAG Template

```
<documents>
  <document index="1">
    <source>filename.pdf</source>
    <document_content>...</document_content>
  </document>
  <document index="2">
    <source>filename2.pdf</source>
    <document_content>...</document_content>
  </document>
</documents>
```

[你的问题放在最后]

Before answering, quote the most relevant passages from each document inside **<quotes>** tags. Then answer based on those quotes.

11.3 Agent Persistence

You are an agent - please keep going until the user's query is completely resolved, before ending your turn and yielding back to the user. Decompose the user's query into all required sub-requests, and confirm that each is completed. Do not stop after completing only part of the request. Only terminate your turn when you are sure the problem is solved.

You must plan extensively before each function call, and reflect on the outcomes of previous calls. Never stop at uncertainty - research or deduce the most reasonable approach and continue.

11.4 The tool uses the intro template

```
<tool_preambles>
- Always begin by rephrasing the user's goal in a clear, concise manner
  before calling any tools.
- Outline a structured plan detailing each logical step.
- As you execute, narrate each step succinctly.
- Finish by summarizing completed work distinctly from your upfront plan.
</tool_preambles>
```

11.5 Parallel Tool Call Template

```
<use_parallel_tool_calls>
If you intend to call multiple tools and there are no dependencies
between them, make all independent calls in parallel. Prioritize
simultaneous calls whenever actions can be done in parallel rather
than sequentially. However, if some calls depend on previous calls'
results to inform parameters, do NOT call those in parallel.
Never use placeholders or guess missing parameters.
</use_parallel_tool_calls>
```

11.6 Contextual Capture Budget Template

```
<context_gathering>
Goal: Get enough context fast. Parallelize discovery and stop as soon
as you can act.

Method:
- Start broad, then fan out to focused subqueries.
- Launch varied queries in parallel; deduplicate paths.
- Avoid over searching.

Early stop:
- You can name exact content to change.
- Top hits converge (~70%) on one area.

Loop: Batch search → minimal plan → complete task.
Search again only if validation fails.
</context_gathering>
```

11.7 Anti-Overengineering Templates

Avoid over-engineering. Only make changes directly requested or clearly necessary.

- Scope: No features, refactoring, or improvements beyond the request.
- Documentation: No docstrings or comments on code you didn't change.
- Defensive coding: No error handling for scenarios that can't happen.
- Abstractions: No helpers for one-time operations. No designing for hypothetical future requirements.

The right amount of complexity is the minimum needed for the current task.

11.8 Self-reflective metaprompts (let the model optimize its own prompts)

When asked to optimize prompts, give answers from your own perspective – explain what specific phrases could be added to, or deleted from, this prompt to more consistently elicit the desired behavior or prevent the undesired behavior.

Prompt: [PROMPT]

The desired behavior is for the agent to [DESIRED]; instead it [ACTUAL]. While keeping as much of the existing prompt intact as possible, what minimal edits would you make to more consistently address these shortcomings?

11.9 Security Confirmation Template

Consider the reversibility and potential impact of your actions. Take local, reversible actions (editing files, running tests) freely. For actions that are hard to reverse, affect shared systems, or could be destructive, ask the user before proceeding.

Examples warranting confirmation:

- Destructive: deleting files, dropping tables, `rm -rf`
- Hard to reverse: `git push --force`, `git reset --hard`
- Visible to others: pushing code, commenting on PRs, sending messages, modifying shared infrastructure

Do not use destructive actions as shortcuts (e.g., `--no-verify`) or discard unfamiliar files that may be in-progress work.

12. Summary: The essence of the two documents

Condense hundreds of pages of material into one sentence: The **key to good use of AI is not to "use clever spells", but to "convey your true intentions and constraints completely, clearly, and structuredly to a collaborator who understands you but doesn't read your mind"**.

More specifically, there are six progressive abilities:

1. **Define success:** You can't tell "what's good," and no one can help you make it "good" — including AI.
2. **Be able to write instructions clearly:** Explicitly formulate the implicit background in your head. The model is a new colleague, not a prophet.
3. **Be able to select the right model and parameters:** fast model for simple execution, inference model for complex judgment; Difficulty and effort match.
4. **Tools and context:** The model itself is the engine, and the tools and context are the fuel and roadmap.
5. **Keep it simple when agentizing:** Start with the simplest enhanced LLM and only add complexity when you really need it.
6. **Continuous testing and iteration:** The evaluation system is the real barrier to AI engineering, not the prompt itself.

All the "prompt engineering techniques" in the past two years—XML tags, few-shots, CoT, thought trees, ReActs, and self-reflection—are the unfolding of the above six principles in different detail scenarios. It is more important to understand the principles than to memorize specific mantras. The model is always changing, and the principles remain the same.

This document is based on the core articles on best practices for AI usage published by OpenAI and Anthropic on their official websites, documentation sites, blogs, and cookbooks between 2024–2026.